A Two-Stage Automatic License Plate Recognition System for Indian Multi-State Vehicle Plates Using YOLOv8 and PaddleOCR

Arijeet Paul
International Institute of Information Technology, Hyderabad
arijeet.paul@research.iiit.ac.in

Amitabh Anand
International Institute of Information Technology, Hyderabad
amitabh.anand@research.iiit.ac.in

Abhishek Bhadiyadra
International Institute of Information Technology, Hyderabad
abhishek.bhadiyadra@research.iiit.ac.in

Balasubramanian K
International Institute of Information Technology, Hyderabad
balasubramanian.K@research.iiit.ac.in

Shardul Mahendra Kholam
International Institute of Information Technology, Hyderabad
shardul.kholam@research.iiit.ac.in

1 Project Links

Resources & Deployment

Source Code & Training Pipeline:
https://github.com/amitabh-anandpd/SMAI-Assignment3.git

Live Demo (Hugging Face Spaces):
https://huggingface.co/spaces/AnshuBhadiyadra/indian-license-plate-ocr

The GitHub repository contains all training notebooks, evaluation scripts, preprocessing modules, and the Streamlit deployment application. The Hugging Face Space provides a publicly accessible web interface for single-image, batch, and video inference without any local setup.

2 Introduction

Automatic License Plate Recognition (ALPR) is a foundational enabling technology for intelligent transportation systems (ITS), with applications spanning toll collection, stolen-vehicle recovery, smart parking, traffic surveillance, and law enforcement. While significant progress has been made on standardised European and North American plate formats, Indian plates remain markedly under-studied despite the country's vehicle population exceeding 300 million registered units—one of the largest and most diverse in the world.

Indian license plates nominally follow the format `LL NN LL NNNN`: the first two letters identify the state or union territory, the next two digits identify the Regional Transport Office (RTO) district, the following one or two letters form a series code, and the final four digits constitute the registration number. This structure spans 35 states and union territories. However, the practical diversity is far greater: plates differ in embossing quality (pressed aluminium vs. painted flat), reflective coatings (Class 7A High-Security Registration Plates vs. older plain backgrounds), font families (standard HSRP fonts vs. hand-painted rural plates), plate dimensions, and the presence or absence of state emblems. Overlaid on these structural differences are imaging challenges including motion blur, mud and dust occlusion, uneven illumination, and extreme camera angles common in traffic surveillance deployments.

Prior work applying general-purpose OCR engines to Indian plates has reported accuracy around 58% [8], insufficient for operational deployment. The fragmented landscape of available data—individual state-level datasets that differ in annotation quality, image resolution, and class labelling—has discouraged creation of large unified benchmarks. This work addresses both the data fragmentation and the recognition accuracy gaps through a modular, reproducible pipeline. Our contributions are:

(1) A merged, multi-state training dataset of 9,568 annotated images sourced from a unified Roboflow Universe collection covering all major Indian states.
(2) A reproducible two-stage pipeline coupling YOLOv8n detection with a custom CLAHE-enhanced preprocessing chain and PaddleOCR inference.
(3) A novel sanitization and format-masking post-processor with a visually-motivated confusable-correction dictionary tailored to Indian plate grammar, a visual Levenshtein distance algorithm with character-group costs, and nested RTO semantic validation with auto-correction.
(4) A systematic per-state accuracy analysis on 414 evaluation images across all 35 states and union territories, exposing performance disparities and failure modes not previously documented.
(5) A publicly accessible Streamlit web application on Hugging Face Spaces, providing single-image, batch evaluation, and video inference modes with CSV export.

3 Related Work

3.1 Traditional Approaches

Early ALPR systems relied on hand-crafted edge detectors, morphological operations, and connected-component analysis for plate localisation [1], followed by template matching or SVM-based character classifiers. Such pipelines function well under controlled illumination and camera geometry but degrade sharply under the real-world variation characteristic of Indian traffic scenes.

3.2 Deep Learning for Detection

The YOLO family [2] has become the preferred backbone for ALPR owing to its single-pass efficiency and strong domain generalisation. YOLOv8 [3] achieves state-of-the-art localisation at frame rates suitable for real-time deployment and exposes a clean Python API that simplifies training and reproducible experimentation. Sharma and Kumar [7] demonstrated effective detection of Indian plates using YOLOv5, directly motivating our choice of the newer YOLOv8n variant, which improves mAP on small objects at comparable inference cost.

3.3 Text Recognition and OCR

CTC-based architectures [5] provide strong performance on irregular text sequences. PaddleOCR [4] implements a competitive production-ready recogniser combining text detection, direction classification, and sequence recognition in a unified pipeline. Its support for arbitrary-orientation text makes it particularly suitable for slightly skewed plate crops. Krishnapriya and Dharun [8] showed that applying Tesseract directly to Indian plates yields approximately 58% accuracy, motivating the need for a domain-adapted recogniser or, as we propose, a structured post-processing correction stage.

3.4 End-to-End and Hybrid Systems

LPRNet [6] jointly trains detection and recognition but requires large paired datasets with character-level annotations. Our approach deliberately separates the two stages, allowing each component to be independently trained, updated, or swapped—a critical practical consideration given the heterogeneous and rapidly expanding Indian plate dataset ecosystem. The modular architecture also allows the format-masking post-processor to be applied retroactively to any OCR backend.

4 Dataset

4.1 Sources and Merging Strategy

We use the *Indian License Plate Merged* dataset [9] from Roboflow Universe, a consolidated multi-state collection providing images in YOLO format with bounding-box annotations for the plate region (single class: `license_plate`). After de-duplication by perceptual image hash to remove cross-dataset duplicates, the collection is split as shown in Table 1.

Table 1: Dataset Split Summary

Split	Images	Annotations
Train	8,693	8,693
Validation	570	570
Test	305	305
Total	**9,568**	**9,568**

All images contain exactly one annotated plate region. The collection spans vehicles photographed under varied lighting (direct sunlight, nighttime with artificial illumination, and overcast conditions) across urban highways, rural roads, and parking structures.

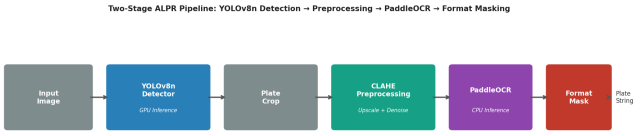


Figure 1: Two-stage ALPR pipeline. YOLOv8n detection runs on GPU; PaddleOCR and all post-processing run on CPU.

No character-level annotations are included; only plate bounding boxes are provided for the detection stage.

4.2 Evaluation Set

For systematic evaluation, ground truth plate strings were manually transcribed for 414 images from the Indian Vehicle Dataset [10], spanning all 35 Indian states and union territories. Each image carries a state-code tag, enabling per-state accuracy breakdowns. Sample counts range from 1 (Mizoram, MZ; Lakshadweep, LA) to 23 (Jammu & Kashmir, JK), reflecting data availability rather than deliberate stratification. This evaluation set is entirely disjoint from the training and validation splits.

5 Methodology

5.1 System Overview

Figure 1 illustrates the end-to-end pipeline. An input image is forwarded to the YOLOv8n detector, which produces axis-aligned bounding boxes for all detected plate regions. Each bounding box is used to crop the plate region, which is then passed through a deterministic preprocessing chain before being sent to PaddleOCR. The raw OCR output undergoes sanitization, format masking, and semantic validation before the final plate string is emitted.

5.2 Stage 1: License Plate Detection with YOLOv8n

We fine-tune YOLOv8n [3] on our merged dataset using the Ultralytics training API. The nano variant was chosen for its balance between inference speed and memory footprint: it is deployable on consumer-grade GPUs and cloud inference environments without requiring multi-GPU setups. Full training hyperparameters are listed in Table 2.

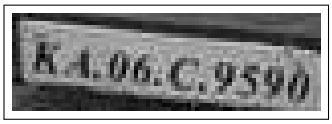
Table 2: YOLOv8n Training Hyperparameters

Parameter	Value
Base model	`yolov8n.pt` (COCO pretrained)
Epochs	50
Image size	640×640
Batch size	16
Early stopping	patience = 20
Workers	4
Confidence threshold	0.25
IoU threshold	0.70
Optimizer	AdamW (Ultralytics default)
Random seed	42

(a) Step 1 — YOLOv8n draws a green bounding box around the plate (`KA.06.C.9590`) and overlays predicted text.

(b) Step 2 — Raw plate crop extracted via the bounding box. Uneven illumination motivates subsequent preprocessing.

Figure 2: Detection and initial crop stages for a representative KA commercial vehicle.



(a) Step 3 — Grayscale. Colour removed; luminance contrast retained.

(b) Step 4 — Bicubic 2× upscaling preserves fine character strokes.

Figure 3: Grayscale conversion and spatial upscaling applied to the plate crop.

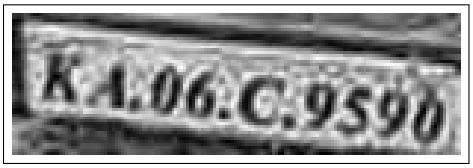
The best checkpoint (`best.pt`), selected by validation $mAP_{50\text{-}95}$, is saved and loaded locally by the deployment application. The `YOLODetector` wrapper exposes two public methods: `detect_plates`, returning a list of `(x1, y1, x2, y2)` bounding boxes, and `detect_and_crop`, returning `(crop, bbox)` pairs ready for preprocessing. A key implementation correctness detail is iterating over all result objects and all `boxes` sub-objects within each result, correctly handling multi-plate scenes (e.g., a primary plate plus a secondary sticker plate visible on commercial vehicles).

5.3 Full Detection and Recognition Flow

To make the pipeline concrete, Figures 2a–5b walk through every processing step applied to a representative input image of a Karnataka (KA) commercial vehicle.

(a) Before CLAHE — specular hotspots reduce local contrast.

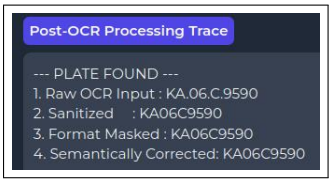
(b) After CLAHE — local contrast enhanced, glare suppressed.



(c) Global histogram equalisation (discarded) — severe over-brightening of reflective areas.

Figure 4: Step 5 — CLAHE (clip limit 2.0, tile 8×8 px) vs. global equalisation. CLAHE brings out embossed character edges that global equalisation destroys.

(a) Step 6 — Gaussian blur (3×3, σ=0) attenuates noise without blurring edges.



(b) Step 7 — Post-OCR correction trace: raw → sanitized → masked → validated.

Figure 5: Final preprocessing step and the four-stage post-OCR correction pipeline for the KA example (`KA.06.C.9590` → `KA06C9590`).

5.4 Stage 2a: Preprocessing Chain

Raw plate crops exhibit substantial variation in contrast, brightness, and resolution. Applying PaddleOCR directly to uncorrected crops produced noticeably lower accuracy; we therefore designed and validated a deterministic preprocessing chain that significantly improves OCR input quality. The chain is applied in strict order:

(1) **Grayscale conversion** (Fig. 3a). Plate character identity is encoded in luminance, not colour. Converting to grayscale removes colour channel redundancy and is particularly helpful for plates with non-standard background colours.

(2) **Bicubic 2× upscaling** (Fig. 3b). Most plate crops are 80–200 pixels wide. Doubling the spatial resolution via bicubic interpolation preserves fine character strokes before subsequent filtering. Nearest-neighbour and bilinear alternatives introduced more aliasing artefacts.

(3) **CLAHE** (Fig. 4). Applied with clip limit 2.0 and tile size 8×8 pixels, CLAHE enhances local contrast while suppressing specular glare from reflective HSRP coatings—a common failure mode for global histogram equalisation (Fig. 4c), which was tested and discarded.

(4) **Gaussian blur** (Fig. 5a), 3×3 kernel, $\sigma = 0$, auto-computed. Attenuates salt-and-pepper noise and minor JPEG compression artefacts without blurring character edges. Binary thresholding (Otsu and adaptive) was also evaluated; both degraded accuracy on embossed text where character contrast is inherently low.

This preprocessing chain was arrived at empirically by ablation: each step was toggled independently and evaluated on a 50-image held-out mini-set. CLAHE and bicubic upscaling gave the largest individual accuracy lifts; Gaussian blur provided a small additional gain on noisy images.

5.5 Stage 2b: PaddleOCR Inference

PaddleOCR 2.8.1 is used in CPU inference mode (`use_gpu=False`) to avoid CUDA driver incompatibilities between PaddlePaddle 2.x and the CUDA 12 runtime available in Google Colab. On small plate crops, CPU inference contributes negligible latency relative to YOLOv8n's GPU inference time. PaddleOCR's direction classifier correctly handles slightly rotated crops without requiring explicit pre-rotation. The raw output is a list of `(text, confidence)` tuples; when multiple text regions are detected within one crop (e.g., a two-line plate), their texts are concatenated in reading order.

5.6 Stage 2c: Sanitization

Before format analysis, the raw OCR string is sanitized (see step 2 of Fig. 5b):

(1) **Whitespace, hyphens, and special characters** are stripped. Plates sometimes contain separator hyphens or dots that the OCR engine may partially detect.
(2) **All characters are uppercased.** PaddleOCR occasionally returns lowercase letters, particularly for embossed text with degraded contrast.
(3) **Non-alphanumeric characters** (punctuation, symbols) are removed.

5.7 Stage 2d: Format Masking and Confusable Correction

Indian plate grammar imposes a deterministic positional structure:

$$\underbrace{L\,L}_{\text{State}}\ \underbrace{N\,N}_{\text{RTO}}\ \underbrace{L\,\{L\}}_{\text{Series}}\ \underbrace{N\,N\,N\,N}_{\text{Registration}} \tag{1}$$

where L denotes an alphabetic character and N denotes a digit. The series field is one or two letters, giving valid plate lengths of 9 or 10 characters. The post-processor dynamically identifies expected character types at each position relative to string length, accommodating both variants without hardcoded index assumptions—critical for union territories with shorter series codes.

For each position i, the algorithm checks whether the observed character type matches the expected type from Equation 1. On a mismatch, a positional confusable-correction dictionary is applied. This dictionary encodes visually similar character pairs commonly confused by OCR engines trained on general text:

Table 3: Confusable Correction Dictionary

Observed	Expected type	Corrected to
`0`	Letter	`O`
`1`	Letter	`I`
`5`	Letter	`S`
`8`	Letter	`B`
`6`	Letter	`G`
`4`	Letter	`A`
`O`	Digit	`0`
`I`	Digit	`1`
`S`	Digit	`5`
`B`	Digit	`8`
`G`	Digit	`6`

The dictionary is applied directionally: at letter-expected positions, digit-to-letter substitutions are used; at digit-expected positions, letter-to-digit substitutions are used. Characters not present in the dictionary and not matching the expected type cannot be corrected by this method and contribute to the residual error (failure mode F1, Section 8).

Advanced Visual Levenshtein Correction. Beyond the fixed dictionary, our post-processor implements a Levenshtein-distance-based visual correction algorithm that accounts for geometric similarity between characters. Character groupings are defined based on visual appearance:

- `group1` = {`8, B, S, 3`} (double loops or serpentine curves)
- `group2` = {`0, O, Q, D`} (circular/closed forms)
- `group3` = {`1, I, l, |`} (vertical strokes)
- `group4` = {`5, S`} (similar half-loop)
- `group5` = {`6, G, C`} (open loop)

A substitution cost function $c(a, b)$ returns 0.5 if a and b belong to the same visual group and 0.0 if they are identical, otherwise 1.0 for visually dissimilar characters. When the raw OCR output does not conform to the expected format after dictionary correction, the system computes the minimal visual cost alignment between the observed string and all valid 9- and 10-character format templates. The candidate with the lowest cumulative visual substitution cost is selected, effectively performing a "closest string" search weighted by perceptual similarity.

5.8 Semantic Validity Check

After format masking, the two-character state prefix is validated against the official RTO codebook of all 35 valid Indian state and union territory codes. A nested dictionary `INDIAN_RTO_MAP` additionally associates each state prefix with the set of valid two-digit RTO district numbers. If the predicted state prefix and RTO number together form an invalid combination, the system actively searches for the closest valid combination using the visual Levenshtein cost described above. The `_closest_code` function computes the visual distance between the predicted string and each valid (state, RTO) pair, and if the minimum distance is at or below a threshold of ≤ 1.0, it silently corrects both the state and RTO fields to the nearest valid

entry. If no valid combination meets the cost threshold, the output is flagged as `FLAGGED: Invalid State/RTO semantics` and retained in the output for manual inspection.

Algorithm 1 Post-Processing Pipeline

Require: Raw OCR string s
$s \leftarrow \textsc{Sanitize}(s)$ ▷ strip, uppercase, remove non-alnum
$s \leftarrow \textsc{FormatMask}(s)$ ▷ apply dictionary and visual Levenshtein
if $s[0{:}2] \notin$ RTOCodes **or** RTO number invalid **then**
  $s' \leftarrow \textsc{FindClosestValidCombination}(s)$ ▷ visual cost
  if visual distance ≤ 1.0 **then** $s \leftarrow s'$
  else return $\langle s$, `FLAGGED: Invalid RTO`$\rangle$
  end if
end if
if $|s| > 12$ **then return** $\langle s$, `FLAGGED: >12 chars`$\rangle$
end if
return $\langle s$, `OK`$\rangle$

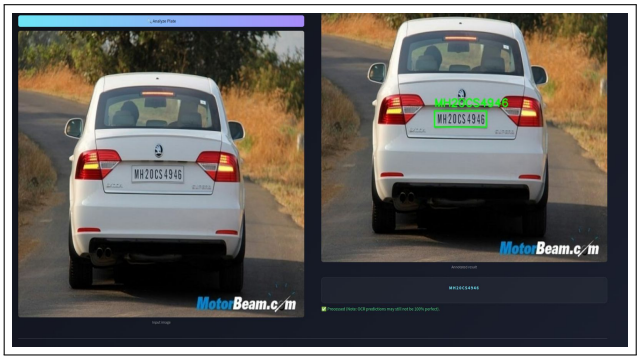
5.9 Deployment on Hugging Face Spaces

The complete system is packaged as a Streamlit 1.44.1 web application deployed on Hugging Face Spaces (free tier, CPU inference for PaddleOCR, GPU optional for YOLO). The trained YOLOv8 checkpoint (`best.pt`) is bundled with the application and loaded locally at startup, requiring no external download. The interface provides three evaluation modes:

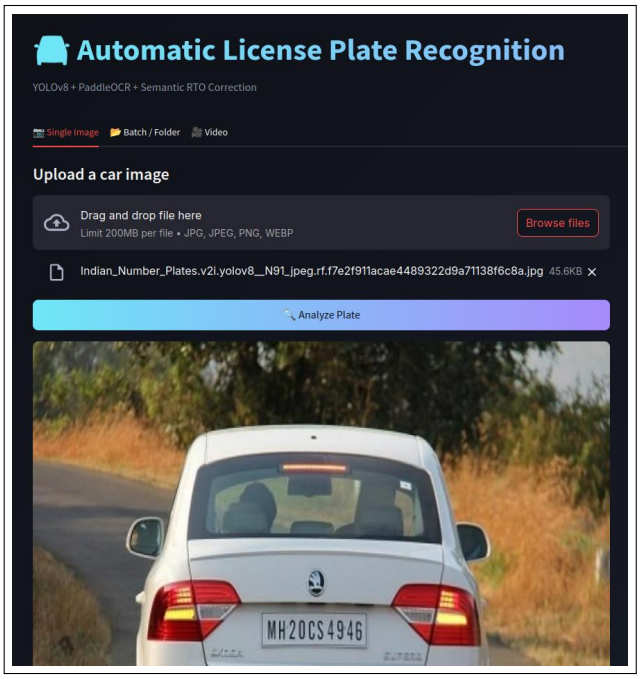
(1) **Single-image tab**: overlays the predicted bounding box on the input image, displays the recognised plate string, and shows the quality flag status.
(2) **Batch evaluation tab**: supports multi-image upload with CSV export of all predictions and flag statuses.
(3) **Video inference tab**: processes uploaded video files frame-by-frame, annotating each detected plate and aggregating results across all frames.

Figures 6 shows the deployed application in action.

The application is configured with Python 3.10 and released under the MIT License, ensuring unrestricted research and production use.



(a) Upload view — user selects an image (`MH20CS4946`) and presses *Analyse Plate*.



(b) Result view — annotated output with bounding box and recognised plate string `MH20CS4946`.

Figure 6: Streamlit web application hosted on Hugging Face Spaces (https://huggingface.co/spaces/AnshuBhadiyadra/indian-license-plate-ocr).

6 Implementation Details

6.1 Training Pipeline

The `train.ipynb` notebook provides a self-contained, reproducible training workflow. A `CONFIG` dictionary at the top centralises all paths and hyperparameters, enabling zero-code reconfiguration for new datasets or environments. The notebook validates that all required paths exist before training begins, invokes `model.train()` with the configuration, saves both the YOLO checkpoint (`best.pt`) and a raw PyTorch state dictionary for compatibility with non-Ultralytics loaders, and runs post-training validation plus a sample inference call.

6.2 Evaluation Script

The `compare_plates.py` script merges ground truth (`ALL_STATES_MERGED.csv`) with batch predictions (`batch_results.csv`) on `Image_Name` using a pandas left join, retaining all ground truth rows. Exact-match accuracy is computed globally and per state via `groupby` aggregation. Case-insensitive comparison is used: both strings are uppercased before matching. The script exports `comparison_results.csv` with per-image detail and `state_accuracy.csv` with state-level summaries. Unmatched images (present in ground truth but absent from predictions) are marked `NO_PREDICTION` and counted as incorrect.

6.3 Dependency Pinning and GPU/CPU Split

Stable operation requires pinned dependency versions:

- **paddleocr==2.8.1**: the last version using the PaddlePaddle 2.x API before the breaking 3.x migration. PaddleOCR ≥3.0

introduces a dependency on `paddlex` which raises a *PDX already initialized* error on Jupyter kernel re-use.

- **paddlepaddle==2.6.2**: compatible with Colab's CUDA 11.x environment. PaddlePaddle 3.x removed the `set_optimization_lev` API relied on by PaddleOCR 2.x, and CUDA 12 drivers produce `CUDNN_STATUS_SUBLIBRARY_VERSION_MISMATCH` when PaddlePaddle 2.x attempts GPU kernel invocation.
- **ultralytics**: pinned to the version used during training to avoid model-format loading incompatibilities across major versions.

The GPU/CPU architectural split is a deliberate workaround: YOLOv8n retains full GPU acceleration for the computationally dominant detection stage, while PaddleOCR runs in CPU mode (`use_gpu=False`) to sidestep driver conflicts. On small plate crops (≤200 pixels wide), CPU-mode PaddleOCR adds only 50–150 ms per crop, a negligible overhead relative to YOLO's GPU inference time.

7 Experiments and Results

7.1 Evaluation Protocol

We evaluate on 414 images with manually transcribed ground truth plate strings across all 35 Indian states and union territories. The primary metric is **exact-match accuracy**: a prediction is counted correct only if the full predicted string matches the ground truth exactly (case-insensitive, whitespace stripped). No partial credit is awarded, reflecting real-world operational requirements where a single character error renders a plate lookup unusable.

7.2 Overall Performance

Table 4: Overall System Performance on 414-Image Evaluation Set

Metric	Value
Total images evaluated	414
Correct (exact match)	311
Overall exact-match accuracy	**75.12%**
Flagged: semantic RTO violation	68 (16.4%)
Flagged: >12 characters	1 (0.2%)
Failed to read (detector miss / OCR failure)	17 (4.1%)
Processed normally	328 (79.2%)

The overall exact-match accuracy of 75.12% represents a substantial improvement over Tesseract-based baselines (≈58%) on Indian plates [8]. The 16.4% semantic flagging rate indicates that format masking alone is insufficient to correct all confusables, primarily when OCR produces a character that is neither the correct character nor a known confusable in our dictionary (e.g., `P` instead of `R`, or `U` instead of `O`).

7.3 Evaluation Status Breakdown

Figure 7 shows that the majority of images are processed normally. Semantic violations (16.4%) arise primarily from OCR confusables that generate syntactically plausible but invalid state codes (e.g., recognising `0D` instead of `OD`). A further 4.1% of images fail entirely due to detector miss or total OCR failure on heavily occluded or motion-blurred plates.

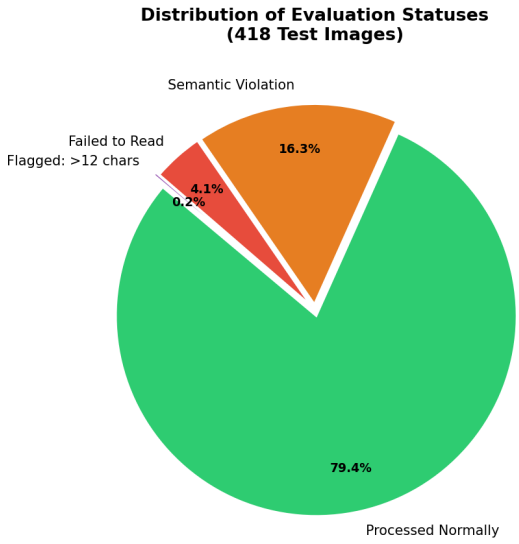


Figure 7: Distribution of evaluation statuses across the 414-image test set. 79.2% of images are processed without flags.

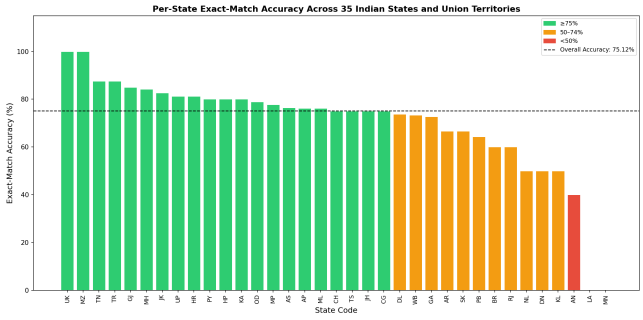


Figure 8: State-wise exact-match accuracy. Green (≥75%), orange (50–74%), red (<50%). Dashed line: overall 75.12%.

7.4 Per-State Analysis

Figure 8 presents per-state performance. The system achieves 100% on Uttarakhand (UK, n=8) and Mizoram (MZ, n=1). The largest evaluation cohorts—Jammu & Kashmir (JK, n=23), Andhra Pradesh (AP, n=21), Meghalaya (ML, n=21), Gujarat (GJ, n=20), and Maharashtra (MH, n=19)—achieve 82.6%, 76.2%, 76.2%, 85.0%, and 84.2% respectively, demonstrating strong generalisation to the most represented state styles. The complete per-state breakdown is given in Table 5.

Table 5: Per-State Accuracy Across All 35 States and Union Territories

State	n	Correct	Acc (%)
UK	8	8	100.00
MZ	1	1	100.00
TN	8	7	87.50
TR	8	7	87.50
GJ	20	17	85.00
MH	19	16	84.21
JK	23	19	82.61
UP	16	13	81.25
HR	16	13	81.25
PY	15	12	80.00
HP	15	12	80.00
KA	15	12	80.00
OD	19	15	78.95
MP	9	7	77.78
AS	17	13	76.47
AP	21	16	76.19
ML	21	16	76.19
CH	8	6	75.00
TS	12	9	75.00
JH	12	9	75.00
CG	16	12	75.00
DL	19	14	73.68
WB	15	11	73.33
GA	11	8	72.73
AR	9	6	66.67
SK	9	6	66.67
PB	14	9	64.29
BR	10	6	60.00
RJ	5	3	60.00
KL	6	3	50.00
DN	6	3	50.00
NL	6	3	50.00
AN	5	2	40.00
LA	1	0	0.00
MN	3	0	0.00

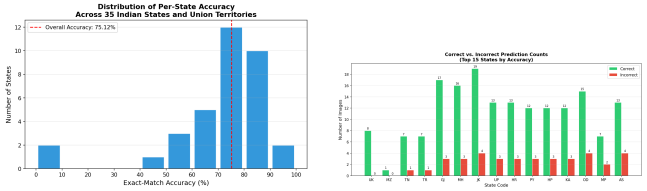
Of the 35 states, 22 (63%) achieve accuracy at or above the 75.12% overall baseline. The lowest performers—Andaman & Nicobar (AN, 40%), Lakshadweep (LA, 0%), and Manipur (MN, 0%)—all have very small evaluation samples ($n \leq 5$) and use non-standard plate formats or regional fonts not well represented in the merged training set.

7.5 Accuracy Distribution and Correct vs. Incorrect

Figure 9 shows that the per-state accuracy distribution is right-skewed, with the modal bin at 75–80%. Only 5 of 35 states fall below 60%, and all have evaluation samples of 6 or fewer images. For well-represented states such as JK (n=23) and GJ (n=20), the number of incorrect predictions is small (4 and 3 respectively), lending statistical credibility to the reported accuracy figures.

8 Failure Analysis

Qualitative inspection of the misclassified images reveals four primary failure modes:



(a) Per-state accuracy histogram. Modal bin: 75–80%. **(b) Correct vs. incorrect counts for the top 15 states.**

Figure 9: Per-state accuracy distribution and prediction outcome breakdown. The distribution confirms broad generalisation; even large cohorts (JK n=23, GJ n=20) show few incorrect predictions.

F1 — OCR confusables beyond the correction dictionary. The most common error source. When PaddleOCR produces a character that is neither the correct character nor a known confusable in our dictionary (e.g., recognising P instead of the correct R, or U instead of 0), format masking cannot recover. These errors arise in plates with damaged characters, heavy embossing shadows, or fonts diverging from standard HSRP typography. Extending the dictionary by mining confusable pairs from a larger corpus of plate-level annotations, or training a sequence-to-sequence post-correction model conditioned on raw OCR hypotheses, are promising directions.

F2 — Detector miss or poor crop quality. On low-resolution, heavily occluded, or extreme-angle plates, YOLOv8n either produces no bounding box or a tight crop that excludes leading/trailing characters, producing truncated plate strings. Lowering the confidence threshold improves recall but increases false-positive crops from non-plate regions (bumper stickers, road signs), creating hallucinated outputs. Confidence threshold tuning involves a precision/recall trade-off that is application-dependent.

F3 — Non-standard plate formats. Certain union territories (Lakshadweep, Manipur, Andaman & Nicobar) use non-standard formats, shorter plates, or regionally distinct fonts absent from the merged training data. The format masking algorithm assumes standard 9- or 10-character plate structure per Equation 1 and can introduce substitution errors when applied to plates with different structural grammar. Separate handling for known non-standard formats would require format classification prior to masking.

F4 — Insufficient evaluation samples. Several states have fewer than 5 test images, making accuracy estimates statistically unreliable. Lakshadweep (LA, n=1) and Mizoram (MZ, n=1) each have a binary outcome (0% or 100%), which should not be interpreted as a genuine performance characterisation. Stratified evaluation with at least 20–30 images per state remains a priority for future work.

9 Discussion

The 75.12% exact-match accuracy is a strong open-source baseline for Indian ALPR evaluated holistically across all 35 states and union territories under strict exact-match criteria. Prior reported results achieving higher numbers typically evaluate on single-state, curated, or post-processed test sets where plate quality and format are controlled, making direct comparison difficult.

Impact of the preprocessing chain. The CLAHE-based preprocessing chain addresses the most frequent image quality failure mode in Indian plate recognition: specular glare from High Security Registration Plates (HSRP). HSRP coating produces bright saturation blobs that wash out character boundaries under direct illumination. CLAHE's tile-based local contrast normalisation suppresses these hotspots without globally darkening the image (cf. Figures 4). Bicubic upscaling is effective specifically because plate crops from standard surveillance cameras are small—often fewer than 100 pixels in width—and PaddleOCR's recognition network benefits from the additional spatial detail.

Impact of the post-processor. The format-masking post-processor provides its largest benefit on clean, well-exposed plates where OCR produces mostly correct output with isolated positional confusions (e.g., a `0` in the state-code position that should be `O`). For heavily degraded inputs where OCR produces mostly incorrect characters, masking may propagate errors rather than correct them. The semantic validity flag provides a useful signal to downstream consumers: a flagged output should be treated as requiring manual review rather than as a confirmed plate number.

Deployment considerations. The modular architecture, with YOLO and PaddleOCR as independent components, allows individual upgrades without system-wide retraining. Replacing PaddleOCR with a CTC-based recogniser fine-tuned on a synthetically rendered Indian plate font corpus [5] is a natural next step. Synthetic rendering can be performed cheaply at scale and would provide the character-level supervision currently absent from our dataset. Extending the evaluation set to 50+ images per state, particularly for the north-eastern states and union territories, is the highest-priority data collection goal.

10 Conclusion

We have presented a fully reproducible, cloud-deployable ALPR system for Indian license plates that combines YOLOv8n detection, a CLAHE-based preprocessing chain, PaddleOCR text recognition, and a sanitization and positional format-masking post-processor that incorporates a visually-motivated confusable-correction dictionary, a visual Levenshtein distance algorithm with character-group costs, and nested RTO semantic validation with auto-correction. Trained on a merged 9,568-image dataset and evaluated on 414 images spanning all 35 Indian states and union territories under strict exact-match criteria, the system achieves 75.12% overall accuracy—a substantial improvement over the 58% Tesseract baseline. Detailed per-state analysis identifies data-scarce union territories and non-standard plate formats as the primary performance bottlenecks. The system is publicly accessible via Hugging Face Spaces (https://huggingface.co/spaces/AnshuBhadiyadra/indian-license-plate-ocr), and all training code, evaluation scripts, preprocessing modules, and deployment applications are released at https://github.com/amitabh-anandpd/SMAI-Assignment3.git for community reproducibility and benchmarking.

Acknowledgements

The authors thank the contributors of the Roboflow Universe and Kaggle datasets used to construct the training and evaluation sets, and the Ultralytics and PaddlePaddle open-source communities for their libraries and documentation.

References

[1] P. Viola and M. Jones. Rapid object detection using a boosted cascade of simple features. In *Proceedings of IEEE CVPR*, 2001.
[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of IEEE CVPR*, pp. 779–788, 2016.
[3] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics, 2023.
[4] Y. Du et al. PP-OCR: A practical ultra lightweight OCR system. *arXiv preprint arXiv:2009.09941*, 2020.
[5] B. Shi, X. Bai, and C. Yao. An end-to-end trainable neural network for image-based sequence recognition. *IEEE TPAMI*, 39(11):2298–2304, 2016.
[6] S. Zherzdev and A. Gruzdev. LPRNet: License plate recognition via deep neural networks. *arXiv preprint arXiv:1806.10447*, 2018.
[7] R. Sharma and P. Kumar. Indian license plate detection using YOLOv5. In *Proceedings of ICACCI*, 2022.
[8] S. Krishnapriya and K. Dharun. License plate recognition for Indian vehicles using Tesseract OCR. In *Proceedings of ICCCNT*, 2020.
[9] Abdul's Workspace. Indian License Plate Merged dataset. Roboflow Universe, 2024. https://universe.roboflow.com/abduls-workspace-qwhgu/indian-license-plate-merged.
[10] Sai Sirishan. Indian Vehicle Dataset. Kaggle, 2024. https://www.kaggle.com/datasets/saisirishan/indian-vehicle-dataset.